Demoze

Anonymous salary and job reviews for Ethiopia. Build spec for the coding agent. Stack: Next.js, Tailwind CSS, Neon Postgres.

Black #0b0b12 / **Purple #7c3aed** / Light #a78bfa

0. How to use this document

Give this whole file to your coding agent and say: **"Read the spec fully. Then build it phase by phase (Section 11). Finish and test one phase before starting the next. Ask me before changing anything in Sections 6 to 8."**

Sections 6, 7 and 8 (privacy, verification, anti-abuse) are the heart of the product. If a shortcut would weaken them, the agent must not take it.

1. Product in one paragraph

A website where people anonymously report what they earn: company, job title, monthly salary in ETB, experience, and an optional review. Visitors search a company or a job title and see real salary ranges and reviews. Trust comes from an optional payslip check (read by AI, then deleted immediately) that earns a Verified badge. The collected data is the long-term asset.

2. Users and goals

User	Goal
Job seeker	Check what a job at a company really pays before applying or negotiating.
Employee	Compare own pay with others in the same role, and share anonymously.
Recruiter / researcher	See market rates by job title, company, city and experience.

3. Tech stack and rules

- **Next.js 15**, App Router, TypeScript (strict). Server Components by default; Server Actions for form posts.
- **Tailwind CSS** only for styling. Mobile-first. Dark mode via prefers-color-scheme.
- **Neon Postgres** through **@neondatabase/serverless** with **Drizzle ORM** and drizzle-kit migrations.
- **Zod** for all input validation (client and server). Never trust client data.
- **AI provider** for payslip reading: build behind one function `extractPayslip(file)` so the provider can be swapped. Key from env var.
- Hosting target: Vercel. No file storage service is used (see Section 7).
- No user accounts, no login, no email collection. Admin area uses one password from env.
- Code quality: small components, no unused packages, clear folder structure, README with setup steps.

4. Folder structure

```
/app
  page.tsx                home: search + latest + top paying roles
```

```
  /company/[slug]/page.tsx  company page: stats, reviews, report button
  /jobs/[title]/page.tsx    salary stats for a job title across companies
  /submit/page.tsx          submission form (+ optional payslip step)
  /about/page.tsx           privacy promise and how verification works
  /admin/page.tsx           moderation (password protected)
  /api/verify/route.ts      payslip upload and AI check
/components                 SearchBar, SalaryRange, ReviewCard, VerifiedBadge, ReportButton
/lib
  db.ts                     Neon + Drizzle client
  schema.ts                 Drizzle tables
  stats.ts                  aggregation and privacy gating
  verify.ts                 extractPayslip + matching logic
  ratelimit.ts              DB-based rate limiting
  normalize.ts              company slug and job title normalization
/drizzle                    migrations
```

5. Database schema

Create these tables with Drizzle. Types shown as SQL for clarity.

```
companies(
  id uuid pk default gen_random_uuid(),
  name text not null,
  slug text unique not null,          -- lowercase, hyphenated
  industry text, city text,
  created_at timestamptz default now()
)

submissions(
  id uuid pk default gen_random_uuid(),
  company_id uuid references companies(id),
  job_title text not null,
  title_normalized text not null,     -- e.g. "software developer"
  monthly_salary_etb integer not null check (monthly_salary_etb between 500 and 2000000),
  bonus_etb integer,
  experience_years smallint not null check (experience_years between 0 and 50),
  employment_type text,               -- full_time | part_time | contract | intern
  city text,
  still_employed boolean default true,
  rating smallint check (rating between 1 and 5),
  review_pros text, review_cons text, -- max 1000 chars each
  verified boolean default false,
  status text default 'published',    -- published | flagged | removed
  device_hash text not null,          -- salted hash, see Section 8
  created_at timestamptz default now()
)

reports(
  id uuid pk default gen_random_uuid(),
  submission_id uuid references submissions(id) on delete cascade,
  reason text not null, details text,
  device_hash text not null,
  created_at timestamptz default now()
)

rate_limits(key text pk, count int, window_start timestamptz)
```

6. Anonymity rules (non-negotiable)

- Do **not** store names, emails, phone numbers, IP addresses or user agents in plain form. Anywhere.
- Do **not** expose exact dates. Show only month and year of submission, or "about 3 months ago".
- Never show a submission's device_hash, id order or anything that links two reviews to the same person.
- **Minimum group size:** show salary stats for a company or job title only when it has **5 or more** published submissions. Below that show "Not enough data yet". Store this as a constant `MIN_GROUP_SIZE = 5`.

- Show salary as ranges (low, median, high) rather than single entries wherever possible. Individual reviews show a salary band (for example 20,000 to 25,000 ETB) not the exact figure.
- Review text is free text: warn users not to include names, team names or details that identify them. Auto-reject text containing phone numbers or email addresses.
- Add a plain-language /about page that explains what is stored and what is not.

7. Payslip verification flow

Verification is optional. Verified submissions get a purple badge and carry more weight in stats display.

- User finishes the form, then sees: "Add a payslip, bank salary SMS or work ID photo to get a Verified badge. We read it and delete it immediately."
- File types: JPG, PNG, PDF. Max 5 MB. Upload goes to `/api/verify` with the submission id.
- The server reads the file **into memory only**. It is **never** written to disk, database, blob storage or logs. Disable request body logging for this route.
- Call `extractPayslip(file)`, which asks the AI model to return strict JSON: company name, job title (if present), monthly amount, currency, month, and an `appears_authentic` boolean with a short reason.
- Matching: company name fuzzy match, and salary within 10 percent of the submitted figure. If both pass and appears_authentic is true, set verified = true.
- After the call, drop all references to the file. Return only {verified: true|false, message}. Never return extracted text to the browser.
- Tell the user to hide account numbers and ID numbers before uploading. Show a clear note on the upload screen.
- Rate limit: 3 verification attempts per device per day.

```
// lib/verify.ts (shape)
export type PayslipResult = {
  company: string | null; title: string | null;
  monthly_amount: number | null; currency: string | null;
  month: string | null; appears_authentic: boolean; reason: string;
};
export async function extractPayslip(file: Buffer, mime: string): Promise<PayslipResult>;
```

8. Anti-abuse and moderation

- **Device hash:** on first visit set an httpOnly cookie with a random id. Store only `sha256(cookie_id + SERVER_SALT)`. This is for limits only, not for tracking people.
- Limits: 1 submission per device per company per 12 months. Max 3 submissions per device per day. Max 10 reports per device per day.
- **Outlier flag:** if a salary is more than 3 times or less than one third of the median for the same job title (when 5 or more exist), set status = flagged and hide until an admin reviews it.
- **Report button** on every review. Reasons: fake or spam, identifies a person, defamatory, wrong company. 3 reports auto-hide the review pending admin check.
- **Admin page** at /admin protected by ADMIN_PASSWORD (httpOnly session cookie). Lists flagged and reported items with Approve, Remove and Edit-company-name actions.
- Honeypot field and a minimum form-fill time of 5 seconds to block simple bots.
- Company names: when a user types a company, suggest existing matches first to prevent duplicates ("Ethio Telecom" vs "Ethiotelecom"). Normalize into a slug.

9. Pages and what they show

Route	Content
/	Big search box (companies and job titles). Below: top paying job titles, recently added companies, and a clear button: "Share your salary". Show total number of submissions as social proof.
/company/[slug]	Company name, overall rating, salary table by job title (low / median / high, count, verified count), reviews list with Verified badges, report button, "Add yours" button. Apply the MIN_GROUP_SIZE rule.
/jobs/[title]	Salary range for one job title across all companies, by years of experience (0-2, 3-5, 6-10, 10+), and top paying companies for that role.
/submit	Step 1 form (company with autocomplete, job title, salary, bonus, experience, type, city, still employed, rating, pros, cons). Step 2 optional payslip upload. Step 3 thank-you screen showing the group's stats, as the reward for sharing.
/about	Privacy promise, how verification works, how to request removal, terms.
/admin	Moderation queue, see Section 8.

10. Design and UX

- Mobile-first: most users are on phones with slow connections. Keep pages light: no large images, no heavy libraries, server-render stats.
- Fonts: load **Noto Sans Ethiopic** with next/font so Amharic text displays correctly, plus a clean Latin font. Use font-display: swap.
- Build all UI text in a dictionary file (`/lib/i18n.ts`) with English now and an Amharic slot ready. Do not hardcode strings in components.
- Clean, trustworthy look: white or dark surface, a black and purple palette (black #0b0b12, purple #7c3aed, light purple #a78bfa, off-white #f5f3ff), large readable numbers. Salary shown as ETB with thousand separators.
- Accessible: labels on all inputs, visible focus states, contrast that passes WCAG AA.
- Loading and empty states for every list. Friendly error messages. No raw errors shown to users.

11. Build phases (give these to the agent one at a time)

Phase 1: Setup

- Create Next.js 15 + TypeScript + Tailwind project. Add Drizzle, Neon serverless driver, Zod.
- Create .env.example with: DATABASE_URL, SERVER_SALT, ADMIN_PASSWORD, AI_API_KEY.
- Write schema.ts, generate and run the migration on Neon.
- Add seed script with 3 fake companies and 12 fake submissions, clearly marked as demo data.

Phase 2: Submit flow

- Build /submit with Zod validation and server action.
- Company autocomplete and slug creation. Job title normalization.
- Device cookie, device_hash, and rate limits.
- Thank-you page that shows group stats (respecting MIN_GROUP_SIZE).

Phase 3: Browse and stats

- Build home search (company and job title).
- Build /company/[slug] and /jobs/[title] with median, low, high, counts.
- Implement stats.ts with the 5-submission privacy gate and experience buckets.
- Add review cards with salary bands and month/year only.

Phase 4: Verification

- Build /api/verify and extractPayslip with the AI provider.
- Memory-only file handling, no logging, delete references after use.
- Matching logic and the Verified badge. Show verified counts in stats.

Phase 5: Safety

- Report button and report storage with the 3-report auto-hide.
- Outlier flagging. Honeypot and minimum fill time.
- Admin page with password login and moderation actions.

Phase 6: Polish and launch

- Amharic font, i18n dictionary, /about page.
- SEO titles and meta for company and job pages. Lightweight Open Graph image.
- Performance pass: test on throttled slow 3G. Fix anything heavy.
- Deploy to Vercel, connect Neon, add env vars, run seed only in preview.

12. Acceptance checklist

- A user can submit a salary in under 60 seconds on a phone with no account.
- No plain IP, email, name or user agent exists anywhere in the database or logs.
- A company with fewer than 5 submissions shows no salary statistics.
- A payslip upload is never saved; confirmed by checking the database, disk and logs.
- A matching payslip returns Verified; a mismatching one does not.
- The same device cannot submit twice for the same company within 12 months.
- Extreme salaries are flagged and hidden until an admin approves them.
- Home page loads fast on throttled 3G and Amharic text renders correctly.
- Admin page is not reachable without the password.

13. Out of scope for the first version

User accounts, company profiles claimed by employers, paid features, mobile apps, interview reviews, and salary alerts. Add these only after the first 500 submissions.

14. Notes for the founder (not for the agent)

- Have a local lawyer read your terms and removal policy before public launch. Companies may dispute reviews, so keep a clear report and takedown process.
- Seed the site with 20 to 30 real submissions from friends and Telegram communities before sharing it widely, because empty sites do not attract visitors.
- Revenue later: paid salary reports and hiring insights for recruiters and HR teams, never selling individual entries.
- Keep the demo seed data clearly labeled and remove it once real data arrives.